

Exact Boolean Decomposition Artifacts under Conditioning: Algebraic Contracts and a Reproducibility Audit

Brian Theory

Research draft prepared for author review

26 September 2026 — Draft 1

Abstract

A compact decomposition of a Boolean truth table is not automatically a useful persistent query representation. We examine what is needed to retain such a decomposition through conditioning and exact model counting. Four familiar families—disjoint XOR factors, binary-field matrix factors, cofactor prototypes up to complement, and coordinate-aligned products—admit direct updates without constructing the entire residual truth table. We give explicit contracts and counting rules, and distinguish their semantic correctness from minimum rank, compact serialization, and source authentication. An executable audit of a supplied implementation reproduces six boundary observations, including unsupported scalar dimensions and metadata that is not independently recomputed. A new reference implementation passes 105,292 exhaustive small-domain conditioning checks. An exploratory experiment uses all 26 outputs of one pinned EPFL control design and 12 synthetic controls. No full-lifecycle timing win over packed truth-table counting is observed at the reported query counts. Some structured synthetic functions save storage, but ideal factor bits, JSON bytes, and raw truth payload bytes give different answers. The result is a bounded methods and audit report, not a claim of new decomposition primitives or general computational superiority.

1 Question, contribution, and scope

Suppose an exact decomposition of a Boolean function has been found once. Later queries fix input variables and ask how many completions make the function true. Must the computation return to the full truth table after every query? For several standard decompositions the answer is no. The harder questions are what the retained representation must contain, how its correctness is checked, and whether its total cost is worthwhile.

The motivating source is a research portfolio around Correspondence Matrices (CMs)[11]. Its structural-computation folder is explicitly a research direction, not a completed submission. The present report separates a small operational problem from that portfolio’s foundational operator calculus and its pair compiler. It does not propose a new meaning for Boolean rank, recast known matrix identities as new CM theorems, or reuse the pair compiler’s historical performance conclusions.

The contribution is threefold. First, the report states a common contract for exact conditioning and counting in a small heterogeneous portfolio. Second, it gives reproducible tests distinguishing reconstruction integrity from stronger claims about rank, size, immutability, and source equivalence. Third, it measures a fully costed reference experiment and preserves negative evidence. The closure facts and elementary counting identities are included with proofs for completeness; historical priority is not claimed for them.

Publication posture. This is a technical-report draft. The computational evidence is deliberately narrow, and the literature does not support presenting “compile once, query repeatedly” or “certified Boolean compilation” as new ideas. Whether the integrated report warrants a research submission remains a separate question from whether its statements and software are useful.

2 Relation to established methods

Knowledge compilation evaluates representations by their succinctness and supported queries and transformations[4]. Conditioning and counting therefore supply an established lens, not a new paradigm. ROBDDs are a central exact symbolic comparator[2]; our implementation of that comparator is explicitly a small fixed-order Python reference, not CUDD.

Disjoint-support decomposition already studies factorizations into functions on separate variable sets[8]. Modern Ashenhurst–Curtis decomposition uses cofactor multiplicity and variable partitions for LUT mapping[10]. Its hardware objective is not identical to our scalar-count workload, but it is essential prior art for claims about discovering useful Boolean structure.

Tensor trains have already been analyzed as Boolean knowledge-compilation representations, with conditioning and counting operations[9]. A factorized tensor with ordinary arithmetic and a factorization over $\mathbb{F}_2$ are not interchangeable objects. Binary matrices have several distinct rank notions[5]. Our matrix multiplication is over $\mathbb{F}_2$, while model counts are ordinary nonnegative integers.

Certified knowledge compilation now includes proof-producing compilation and formally verified checking and counting[3]. A Python reconstruction check does not match that assurance level. Finally, algebraic model counting provides a broader semiring setting[7]; weighted variants are not an empirical contribution of this draft.

3 Source recovery and the operational boundary

The supplied archive and its nested archives were inventoried with SHA-256 hashes and CRC checks. This inventories 3,668 file entries, including nested archive files, with 1,228 distinct content hashes. It does not imply that every line of every neighboring manuscript or log received a substantive audit. The reading map identifies the primary structural notes, the relevant foundation theorem, and the recovered implementation.

The principal code examined is `cmbench/recognition/gf2_decomposition.py` in the recovered `paper_b_current_01/source` snapshot. Its four artifact tags are `xor_components`, `gf2_rank`, `cofactor_blocks`, and `kronecker`. The file advertises $2 \leq n \leq 10$ and a default limit of 64 candidate partitions. The partition family combines an interaction-component proposal, singleton partitions, and further balanced candidates; it is not a global search over every possible decomposition. A minimum returned by the selector means a minimum among generated candidates under its stated score.

The source’s rank cost $r(R + C)$ is an ideal payload count. It is not measured JSON size or resident memory. Source hashes and reconstruction checks are useful integrity mechanisms, but the scope of their assurance must be stated explicitly.

The reference implementation in this report is new code, not an unannounced patch of the historical compiler. Its product artifact uses disjoint table factors. A coordinate-aligned matrix Kronecker product can be represented this way by grouping each factor’s row and column variables. The new discovery procedure tests simple rank-one product splits; it does not reproduce every

historical Kronecker search rule. Comparisons concern semantic families, not identical discovery coverage.

4 Definitions and exact artifact semantics

Let $V = (v_0, \dots, v_{n-1})$ be an ordered list of distinct declared variables and $f : \{0, 1\}^V \rightarrow \{0, 1\}$. The first listed variable is the most significant coordinate of an assignment index. The stored truth integer has bit a equal to $f(a)$. Declared variables are not silently removed when they happen to be irrelevant to the function: they still contribute assignments to a model count.

A partial assignment $\rho : T \rightarrow \{0, 1\}$, $T \subseteq V$, gives a residual function $f|_\rho$ on $V \setminus T$ in inherited order. Define

$$N(f, \rho) = |\{a \in \{0, 1\}^{V \setminus T} : f(a, \rho) = 1\}|. \quad (1)$$

The empty variable set has one assignment. Thus a scalar true function has count one, not zero. Fully assigning a function and querying an unassigned constant are required boundary cases.

Definition 4.1 (Exact persistent artifact). An artifact consists of a declared ordered variable universe, a representation tag, and a finite payload with an evaluation map. It is exact for f when its scalar evaluation equals f on every declared assignment. A conditioning operation is correct when its output has the residual variable universe and evaluates to $f|_\rho$. A counting operation returns the integer in (1).

We use four families. In each, the factor scopes or the two matrix scopes form a disjoint cover of the declared universe.

Disjoint XOR. The payload is a bit c and local truth tables g_i on disjoint blocks V_i , with

$$f = c \upharpoonright g_1 \upharpoonright \dots \upharpoonright g_k. \quad (2)$$

Empty blocks are permitted after conditioning. The empty product convention used below also covers scalar artifacts.

Binary-field matrix factors. A cut $X \mid Y$ yields a matrix $M_{x,y} = f(x, y)$ with $R = 2^{|X|}$ rows and $C = 2^{|Y|}$ columns. The payload stores

$$M = UV, \quad U \in \mathbb{F}_2^{R \times r}, \quad V \in \mathbb{F}_2^{r \times C}. \quad (3)$$

Here r is an inner dimension. It equals the matrix rank only when minimality has been established. Rank zero is allowed. The standard relationship between GF(2) rank and XOR sums of separated products is already stated in the neighboring foundations manuscript[12]; it is background here.

Cofactor prototypes. For each row index x , store a prototype index $j(x)$ and flip bit $e(x)$, so

$$M_{x,y} = p_{j(x)}(y) \upharpoonright e(x). \quad (4)$$

Prototypes can be canonicalized up to complementation. Storing a prototype table together with references is different from storing the general functional encoding produced by an ACD optimizer.

Disjoint products. For tables g_i on disjoint blocks, store $f = \prod_i g_i$. Since factors are Boolean, multiplication is conjunction. A matrix identity $M = A \otimes B$ has this form after the factor variables are grouped consistently. Arbitrary row permutations and arbitrary row subsets need not preserve the displayed Kronecker shape.

5 Conditioning without a full residual table

Theorem 5.1 (Family-wise conditioning). *Each family above admits an exact conditioning operation which uses only its factors and the assignment ρ . It need not construct the $2^{n-|T|}$ entries of the complete residual truth table. Scalar and one-variable residuals are included.*

Proof. For disjoint XOR and product artifacts, replace each local factor g_i by its restriction to $\rho|_{V_i}$. The defining equality is preserved under substitution. Keep the residual local scope even when the restricted table is constant, or account explicitly for any discarded free variables.

For a rank artifact, let I and J be the row and column assignments compatible with the restrictions on X and Y . The residual matrix is

$$M[I, J] = U[I, :]V[:, J]. \quad (5)$$

The equality follows entry by entry from the original matrix product. Only coefficient rows and basis-column selections are needed.

For prototypes, retain the references for rows in I and restrict every referenced prototype to columns in J . Equal or complementary restricted prototypes may be merged with the flip bits adjusted. This changes the dictionary, not the represented row.

For $A \otimes B$, a Boolean coordinate restriction induces Cartesian selections in the grouped row and column coordinates of each factor. Consequently it selects submatrices of A and B and retains their product. This assertion is made only with the stated coordinate alignment. The disjoint-product representation gives the same argument directly. $\square$

Proposition 5.2 (Basic monotonicity). *Under the explicit updates in the theorem, the stored number of XOR or product factors can be kept constant, $GF(2)$ matrix rank cannot increase, and the number of used prototype classes up to complement cannot increase. If rank factors retain their old inner dimension, their ideal matrix payload $r(R' + C')$ cannot increase.*

Proof. The factor-list update changes tables rather than adding factors. Matrix restriction multiplies by row and column selection matrices, so its rank does not increase. Every new prototype class is the image of an old class under the same column restriction, possibly with classes merged. Finally $R' \leq R$ and $C' \leq C$. $\square$

This proposition says nothing about the size of a JSON audit history, allocator overhead, or the number of factors after a fresh decomposition search. Those are different quantities. Nor does it promise that a residual rank artifact remains the smallest representation in the portfolio.

Example 5.3 (A count-preserving local update). Let $f(x_1, x_2, x_3, x_4) = (x_1x_2) \upharpoonright (x_3x_4)$. Each factor has one satisfying assignment among four, so f has six satisfying assignments among sixteen. Under $x_1 = 0$, the first factor is the constant zero on the still-free variable x_2 . Its zero count is two. The residual function therefore has two satisfying assignments, not one. Dropping x_2 from the declared universe without a multiplicity correction would be a counting error.

6 Direct exact counts

Theorem 6.1 (XOR, product, and prototype counts). *Suppose the factors have already been conditioned. Write z_i, o_i for the numbers of zero and one assignments to the i th factor, and $t_i = z_i + o_i$. Then*

$$N \left(c \upharpoonright_i g_i, \emptyset \right) = \frac{\prod_i t_i - (-1)^c \prod_i (z_i - o_i)}{2}, \quad (6)$$

$$N \left(\prod_i g_i, \emptyset \right) = \prod_i o_i. \quad (7)$$

For a prototype artifact with C' residual columns, count

$$\sum_{x \in I} \begin{cases} \text{wt}(p_{j(x)}|_J), & e(x) = 0, \\ C' - \text{wt}(p_{j(x)}|_J), & e(x) = 1. \end{cases} \quad (8)$$

Proof. For a Boolean bit b , $(-1)^b$ is 1 on zero and -1 on one. Disjoint factor scopes make the sum of the sign of (2) factor into $(-1)^c \prod_i (z_i - o_i)$. This sign sum equals the number of zero assignments minus the number of one assignments; the total is $\prod_i t_i$. Solving the two equations gives (6). A product is one exactly when all factors are one, giving (7). A complemented prototype has $C' - \text{wt}(p)$ ones, giving (8). $\square$

Rank factors require parity-aware counting. It is invalid to multiply the ordinary integer sums of U and V : the matrix product in (3) is over $\mathbb{F}_2$. Our reference algorithm groups equal coefficient rows of $U[I, :]$. For each distinct coefficient vector, it XORs the selected basis rows, takes a packed popcount, and multiplies by that coefficient row’s multiplicity. It can hold one packed residual row at a time instead of the whole $R' \times C'$ matrix. This is an exact algorithm but not a theorem that low rank makes every query fast. A more detailed profile-based complexity analysis is a natural next revision.

Cost and output contracts. For explicit local tables, restriction can scan their entries rather than all global assignments. Rank slicing touches at most the coefficient and basis tables, and counting can process distinct coefficient rows. A scalar count and an explicit residual truth vector are different outputs: writing the latter already requires $2^{n-|T|}$ output bits. Avoiding this output does not itself establish an asymptotic or measured advantage over another scalar-count representation.

Decision queries. Once the count is exact, satisfiability is the test $N > 0$ and tautology is $N = 2^{|V \setminus T|}$. No separate SAT or validity experiment is claimed.

7 What the archived loader actually certifies

The audit preserves the unmodified source and executes small witness mutations. Table 1 summarizes the outcomes. “Admitted” means that the archived loader accepted the modified document; it does not mean that the original producer generated that document in ordinary use.

The first two observations separate an exact semantic factorization from a trusted optimization score and a minimum-rank assertion. The third is the ordinary distinction between a self-consistent digest and externally rooted source equivalence. The fourth limits claims of immutable persistence.

Observation	Reproduced consequence
H1: payload cost	A witness with actual producer score 16 accepts declared <code>factor_bits=1</code> after the document digest and ratio are updated.
H2: rank metadata	Appending a zero basis row and increasing the declared rank preserves the truth table and passes admission.
H3: source identity	Changing a basis bit and replacing the bundled source digest with the changed truth’s digest passes admission. No external source was supplied to this loader.
H4: mutability	A frozen dataclass contains a mutable dictionary. Changing a nested basis value after admission changes reconstruction.
H5–H6: residual boundary	Public analyzer calls with $n = 0$ and $n = 1$ are rejected.

Table 1: Contract boundaries of the archived implementation, not evidence that ordinary producer outputs were semantically wrong.

The last two show that a mathematical closure theorem cannot simply be attributed to the existing schema.

The new scalar checker consumes an externally supplied truth integer and evaluates the witness independently on every assignment. It does not import the producer or the direct query evaluator. This reduces shared implementation logic but is not formal verification and does not eliminate common specification mistakes. Nested tuples are used in the new payloads. Serialization is not a proof certificate, and the decoder is not advertised as a hardened adversarial-input service.

8 Exploratory experiment

8.1 Frozen endpoint and data

The measured task is an integer model count under a partial assignment, not construction of a residual table. The common input is a truth integer. Circuit-to-truth extraction is performed before timing and is common to all methods; it is not claimed free in an end-to-end circuit pipeline.

All 26 primary outputs of one EPFL control benchmark are used[6]. Its exact pinned source SHA-256 is recorded in the supplement and matches the historical corpus metadata. The source was recovered through a text view and its bytes were hash-checked. Scalar recursive and packed cube evaluations independently agree on the extracted functions, including off-set covers and the constant output. The declared variables of an output are its syntactic cone inputs in source order. This is one related design, not 26 independent benchmark families.

Twelve synthetic controls use parity, inner product, products of pairwise XORs, and pseudorandom truth tables at $n = 4, 8, 12$. Their structure is intentional. They test recognition and query behavior, not natural incidence. For each case, 128 partial assignments are generated with a fixed seed; each variable is fixed with probability one half and then assigned a uniformly chosen bit. These are independent views of the original function, not a cumulative restriction chain.

8.2 Search, baselines, and lifecycle cost

The new search examines at most 16 prescribed partitions, plus an ANF-component XOR artifact and a flat fallback. It chooses the smallest compact JSON serialization, with deterministic tie-breaking. This policy was frozen before the first timed run. A different encoding or cost function can select a different winner.

The packed baseline stores the source truth integer and prepares literal masks. A query intersects

the applicable masks and uses native integer popcount. The second baseline is a fixed-order reference ROBDD with reduction and memoized exact counting; it is not an optimized CUDD implementation. The attempted CUDD-binding installation failed because external binary downloads were unavailable in the runtime. DSD, ACD, optimized tensor-train, and industrial KC baselines were not executed. Their absence prevents a comparative state-of-the-art claim.

For each method, five repetitions rotate method order. The timed setup includes construction or discovery, source-equivalence checking when the representation changes, serialization and parse time, and mask initialization where applicable. The artifact arm constructs a conditioned artifact and then counts it; the other baselines count directly. This asymmetry is visible and motivates a fused-query sensitivity analysis rather than a claim about optimal algorithms.

With setup S and average warm query cost t , the descriptive lifecycle model is

$$T(Q) = S + Qt. \quad (9)$$

Results are reported at $Q = 1, 10, 100, 1000$. A finite estimated break-even against baseline (S_b, t_b) requires $t < t_b$ when $S > S_b$, and is then approximately $(S - S_b)/(t_b - t)$. It is not legitimate to infer amortization merely from a compact factor payload.

8.3 Results and their limits

All tested methods returned identical counts. The minimum-JSON selector retained the flat representation on every one of the 26 external outputs. Among the synthetic controls it selected four XOR artifacts, two product artifacts, and six flat representations. No artifact pipeline had a lower estimated full-lifecycle cost than the packed baseline at $Q = 1000$; the raw per-case data include the other query counts and all repetitions.

Case at $n = 12$	Selected family	JSON bytes	Raw truth bytes
Parity	XOR	191	512
Inner product	XOR	155	512
Product of pairwise XORs	Product	157	512
Pseudorandom control	Flat	1324	512

Table 2: Storage quantities from the frozen reference experiment. Raw truth bytes exclude a container header. JSON and raw payload are deliberately not conflated.

For example, the $n = 8$ parity artifact occupies 147 JSON bytes, compared with 32 raw truth payload bytes: an ideal structural factorization does not automatically make that container smaller. At $n = 12$, three structured controls do use fewer JSON bytes than the raw truth payload (Table 2). This is an encoding-specific observation, not a lower bound for all compressed formats.

The aggregate query-time ratio is not a useful speedup headline: 32 selected artifacts are flat fallbacks, so most ratios compare nearly the same packed query with extra discovery before it. Nonflat artifacts are slower in this Python condition-then-count implementation. No confidence interval over an industrial population is justified, and no broad failure of compressed computation follows from these measurements.

9 Verification coverage and reproducibility

The exhaustive core suite enumerates every Boolean function for $0 \leq n \leq 3$: $2 + 4 + 16 + 256 = 278$ functions. Across the generated representations it verifies 3,988 artifacts and serialization round

trips, rejects 3,988 mismatched external truth vectors, and checks 105,292 conditioned artifact/count pairs. All passed. Exhaustive coverage is restricted to that domain; larger benchmark cases are individually verified against their source truth integers.

The supplement contains the original archive, the unmodified recovered code, source hashes, the new producer, separate scalar checker, dataset parser, baselines, frozen protocol, raw timing samples, and machine-readable results. No historical P14 speedup is promoted into this experiment. Historical logs that were not replayed retain their historical status.

A fresh reader can run the core tests without third-party dependencies. The reference implementation caps discovery at sixteen variables, but the measured synthetic functions stop at twelve. This is not evidence of useful scaling to arbitrary Boolean formulas.

10 Portfolio boundary and next research gate

The foundations manuscript remains responsible for the CM/LM calculus, logical pairing, higher-arity lifting, signed frames, and the standard rank/separability discussion[12]. The pair-compiler paper retains its compiler rules, S/T/H provenance, and P14 evidence. The present report neither duplicates those results as new contributions nor depends on a new CM algebra.

The small operator-containment lattice, sixteen-operator orbits, and structural retrieval ideas are omitted. They do not strengthen this particular conditioning/counting argument. Retrieval would require a separate independently meaningful task and contemporary comparators, not synthetic labels that simply recover their own mutation generator.

The next decision should be made on evidence. Useful extensions include a rank-profile counting analysis, tighter relationships between explicit cofactor dictionaries and GF(2) factors, a compact binary encoding, fused count queries, stronger external corpora, and optimized baseline implementations. A standalone technical report is currently defensible. A distinct research-paper novelty claim and competitive systems result are not yet established.

11 Conclusion

A semantic decomposition, a persistent data structure, an optimization score, and a source-equivalence certificate are different objects. Four classical structural families can support exact conditioning and counting without constructing the global residual table, provided scopes, scalar cases, and algebra are explicit. That statement survives exhaustive small-domain testing in a new implementation. It does not imply that the historical schema already has the required closure or that the method is faster than packed truth-table counting. The present evidence supports a bounded and reproducible methods/audit report, with the strongest publication claims deliberately withheld.

Process disclosure. This research draft and its executable checks were prepared with AI assistance. Subsequent specialist-role audit reports are internal role-separated analyses, not external peer review or independent human endorsement. All unresolved correctness, priority, and generalization questions remain visible in the accompanying ledgers.

References

- [1] J. Alman and K. Rao. *Faster Walsh-Hadamard and Discrete Fourier Transforms From Matrix Non-Rigidity*. arXiv:2211.06459, version 2, 2023. <https://arxiv.org/abs/2211.06459>.

- [2] R. E. Bryant. Graph-Based Algorithms for Boolean Function Manipulation. *IEEE Transactions on Computers*, C-35(8):677–691, 1986. <https://www.cs.cmu.edu/~bryant/pubdir/ieeetc86.pdf>.
- [3] R. E. Bryant, W. Nawrocki, J. Avigad, and M. J. H. Heule. *Certified Knowledge Compilation with Application to Formally Verified Model Counting*. arXiv:2501.12906, 2025. <https://arxiv.org/abs/2501.12906>.
- [4] A. Darwiche and P. Marquis. A Knowledge Compilation Map. *Journal of Artificial Intelligence Research*, 17:229–264, 2002. <https://arxiv.org/abs/1106.1819>.
- [5] D. DeSantis, E. Skau, D. P. Truong, and B. Alexandrov. *Factorization of Binary Matrices: Rank Relations, Uniqueness and Model Selection of Boolean Decomposition*. arXiv:2012.10496, version 2, 2021. <https://arxiv.org/abs/2012.10496>.
- [6] Integrated Systems Laboratory, EPFL. *EPFL Combinational Benchmark Suite*. Pinned commit 0060e156826e733d69bf5b3322d1bdd0d03a1f9a; accessed 26 September 2026. MIT license. <https://github.com/lsils/benchmarks>.
- [7] A. Kimmig, G. Van den Broeck, and L. De Raedt. *Algebraic Model Counting*. arXiv:1211.4475, 2012. <https://arxiv.org/abs/1211.4475>.
- [8] A. Mishchenko. *An Approach to Disjoint-Support Decomposition of Logic Functions*. Author-hosted technical report, Portland State University, 2001. https://people.eecs.berkeley.edu/~alanmi/publications/2001/tech01_dsd.pdf.
- [9] R. Onaka, K. Nakamura, M. Nishino, and N. Yasuda. Tensor Decomposition Meets Knowledge Compilation: A Study Comparing Tensor Trains with OBDDs. *Proceedings of AAAI*, 39(14):15109–15117, 2025. DOI: 10.1609/aaai.v39i14.33657.
- [10] A. Tempia Calvino, A. Mishchenko, G. De Micheli, and R. Brayton. *Practical Boolean Decomposition for Delay-driven LUT Mapping*. arXiv:2406.06241, 2024. <https://arxiv.org/abs/2406.06241>.
- [11] B. Theory. *CM Structural Computation and Decomposition: Research Portfolio*. User-supplied archive, `PapersforPublication(2).zip`, 2026. Original bytes and SHA-256 inventory are preserved in this report’s supplement.
- [12] B. Theory. *Correspondence and Logical Matrices: A Boolean Operator Calculus*. Unpublished supplied manuscript, 2026. Rank/separability theorem, lines 1018–1044 of the preserved TeX source.